

Universidad de Concepción

Facultad de Ingeniería
Departamento de Ingeniería Informática y Ciencias de la Computación

Estructuras de Datos y Algoritmos Avanzados (2026-2)

Boletín 03: String Matching

Autor:
Bryan Eliseo Aguirre Fuentes

Matrícula:
2024443402

Profesor:
Dr. José Fuentes

Ayudante:
Leonardo Lovera

Concepción, Chile
6 de noviembre de 2026

Índice

1. Resumen **1**
1.1. Propósito y alcance del estudio . . . 1
1.2. Estructuras y algoritmos estudiados . . . 1

2. Análisis teórico **1**
2.1. Búsqueda por fuerza bruta . . . 1
2.2. Knuth–Morris–Pratt (KMP) . . . 2
2.3. Rabin–Karp . . . 2
2.4. Suffix Array . . . 2
2.5. Comparación de complejidades . . . 3

3. Implementación **3**
3.1. Búsqueda por fuerza bruta . . . 3
3.2. Knuth–Morris–Pratt (KMP) . . . 3
3.3. Rabin–Karp . . . 3
3.4. Suffix Array . . . 4

4. Metodología experimental **4**
4.1. Características de la máquina . . . 4
4.2. Datos de entrada . . . 4
4.3. Experimentos e hipótesis . . . 4
4.3.1. Efecto del largo del texto . . . 5
4.3.2. Efecto del largo del patrón . . . 6
4.3.3. Efecto del número de ocurrencias . . . 6

5. Resultados experimentales **7**
5.1. Experimento 1: largo del texto . . . 7
5.2. Experimento 2: largo del patrón . . . 8
5.3. Experimento 3: número de ocurrencias . . . 9
5.4. Resumen numérico de tiempos y memoria . . . 10

6. Conclusiones **11**
6.1. Experimento 1: efecto del largo del texto . . . 11
6.2. Experimento 2: efecto del largo del patrón . . . 11
6.3. Experimento 3: efecto del número de ocurrencias . . . 11
6.4. Conclusión general . . . 12

Referencias **13**

1. Resumen

1.1. Propósito y alcance del estudio

Este estudio compara experimentalmente cuatro métodos de búsqueda de patrones en texto: búsqueda por fuerza bruta, Suffix Array, Knuth–Morris–Pratt (KMP) y Rabin–Karp. La fuerza bruta se incluye como **línea base** para contextualizar el desempeño de los métodos estudiados. Para efectos de los experimentos, *búsqueda* significa encontrar y contar todas las ocurrencias del patrón en el texto, no devolver sus posiciones. Se analiza cómo influyen el largo del texto, el largo del patrón y el número de ocurrencias en el tiempo de búsqueda; además, se consideran el costo de construcción del índice y el consumo de memoria. Los resultados se interpretan junto con las complejidades teóricas para identificar los escenarios en que conviene cada alternativa.

1.2. Estructuras y algoritmos estudiados

Fuerza bruta: Revisa cada posición posible del texto y compara desde allí los caracteres con el patrón. No requiere preprocesamiento y sirve como referencia sencilla, aunque puede repetir comparaciones.

Suffix Array: Ordena las posiciones iniciales de los sufijos del texto. Para buscar un patrón, localiza mediante búsqueda binaria el intervalo de sufijos que comienza con él y cuenta sus ocurrencias. Su índice requiere una etapa de construcción previa.

KMP: Busca el patrón recorriendo el texto y usando información de prefijos del patrón para evitar comparaciones repetidas. Su búsqueda tiene tiempo lineal en el largo del texto y del patrón, sin requerir un índice persistente.

Rabin–Karp: Compara primero valores hash del patrón y de ventanas del texto; cuando coinciden, verifica los caracteres para confirmar la ocurrencia. El hash de las ventanas se actualiza incrementalmente y su desempeño depende, entre otros factores, de las colisiones.

2. Análisis teórico

Sea T el texto y P el patrón. Denotamos sus longitudes por $n = |T|$ y $m = |P|$, respectivamente. Excluimos el patrón vacío ($m = 0$), cuyo número de coincidencias depende de la convención adoptada. Analizamos el caso usual $1 \leq m \leq n$; si $m > n$, las implementaciones devuelven cero inmediatamente. Por *búsqueda* entendemos contar todas las ocurrencias, incluidas las superpuestas. Usamos notación $O(\cdot)$ para expresar cotas superiores y $\Theta(\cdot)$ cuando la cota es ajustada. El espacio se expresa en palabras y no incluye la entrada. Como los métodos devuelven un contador, el espacio de salida es $O(1)$; si devolvieran todas las posiciones, habría que sumar $O(z)$, donde z es el número de ocurrencias.

2.1. Búsqueda por fuerza bruta

El algoritmo prueba cada posición de T y compara desde allí los caracteres de P, avanzando una posición tras cada intento.

- **Preparación:** $\Theta(1)$; no construye estructuras previas.
- **Mejor caso de búsqueda:** $\Theta(n - m + 1)$, cuando cada intento falla al comparar el primer carácter. Una coincidencia inicial puede añadir hasta $O(m)$ comparaciones, pero el algoritmo debe revisar las posiciones restantes para contar todas las ocurrencias.
- **Peor caso de búsqueda:** $\Theta((n - m + 1)m)$, si cada ventana coincide en casi todo el patrón antes de fallar o confirmarse.

- **Memoria auxiliar:** $\Theta(1)$.

2.2. Knuth–Morris–Pratt (KMP)

KMP construye una tabla que indica qué prefijo del patrón puede conservarse después de una discrepancia. Así desplaza el patrón sin retroceder en el texto. La implementación construye la tabla en cada llamada y luego recorre el texto; tras una coincidencia, también usa la tabla para conservar posibles coincidencias superpuestas.

- **Preparación e inicialización:** $\Theta(m)$ tiempo y memoria para construir la tabla de prefijos.
- **Mejor caso de búsqueda:** $\Theta(n)$.
- **Peor caso de búsqueda:** $\Theta(n)$; el recorrido avanza por el texto y los retrocesos del patrón son amortizadamente lineales. Incluida la preparación, el tiempo es $\Theta(n + m)$.
- **Memoria auxiliar:** $\Theta(m)$, por la tabla.

2.3. Rabin–Karp

Rabin–Karp calcula el hash del patrón P y el de la primera ventana de T durante la preparación. La búsqueda verifica esa primera ventana y luego desplaza la ventana, actualizando su hash en tiempo constante. Compara caracteres para confirmar cada coincidencia de hash.

- **Preparación e inicialización:** $\Theta(m)$ tiempo para calcular ambos hashes: el de P y el de la primera ventana de T.
- **Mejor caso de búsqueda:** $\Theta(n - m + 1)$, cuando el hash de P difiere del hash de todas las ventanas examinadas. No hay verificaciones carácter a carácter y cada ventana se procesa en tiempo constante.
- **Peor caso de búsqueda:** $\Theta((n - m + 1)m)$, si muchas ventanas requieren una verificación completa, por colisiones o coincidencias reales superpuestas.
- **Memoria auxiliar:** $\Theta(1)$.

El peor caso puede ocurrir por colisiones de hash o por coincidencias reales en muchas ventanas. Como se usan parámetros de hash fijos y no se supone una distribución aleatoria de las entradas, no se afirma una garantía de tiempo esperado. Se asume que las operaciones con el hash modular cuestan tiempo constante.

2.4. Suffix Array

La estructura ordena los sufijos de T mediante duplicación de prefijos: en cada ronda duplica el largo del prefijo ordenado y actualiza clases con conteo. Para contar un patrón, realiza dos búsquedas binarias que delimitan los sufijos que comienzan con P; no recorre individualmente las z ocurrencias.

- **Preparación (construcción):** $O(n \log n)$ tiempo en el peor caso y $\Theta(n)$ memoria, mediante duplicación de prefijos y ordenamiento por conteo en cada ronda. Esta implementación ejecuta todas las rondas, por lo que su construcción toma $\Theta(n \log n)$.
- **Mejor caso de búsqueda:** $\Theta(\log n)$, si las comparaciones binarias terminan al revisar el primer carácter.
- **Peor caso de búsqueda:** $O(m \log n)$, pues cada comparación puede revisar hasta m caracteres. Es la búsqueda binaria básica, sin optimizaciones basadas en LCP; contar el intervalo cuesta $O(1)$.
- **Memoria:** $O(1)$ auxiliar por consulta; el índice persistente ocupa $\Theta(n)$ y la memoria temporal de construcción, $\Theta(n)$.

La memoria depende de los largos de entrada, no de su contenido; por ello no cambia entre mejor y peor caso.

2.5. Comparación de complejidades

Los tiempos de búsqueda incluyen el conteo. En Rabin–Karp, la preparación incluye el hash de P y el de la primera ventana de T. La preparación del Suffix Array corresponde a su construcción; se distingue el espacio auxiliar por consulta del índice persistente y de la memoria temporal de construcción.

Método	**Preparación**	**Mejor búsqueda**	**Peor búsqueda**	**Memoria auxiliar / índice**
Fuerza bruta	$\Theta(1)$	$\Theta(n-m+1)$	$\Theta((n-m+1)m)$	$\Theta(1)$
KMP	$\Theta(m)$	$\Theta(n)$	$\Theta(n)$	$\Theta(m)$
Rabin–Karp	$\Theta(m)$	$\Theta(n-m+1)$	$\Theta((n-m+1)m)$	$\Theta(1)$
Suffix Array	$\Theta(n \log n)$	$\Theta(\log n)$	$O(m \log n)$	$O(1)$ consulta; $\Theta(n)$ índice persistente; $\Theta(n)$ temporal al construir

3. Implementación

Los algoritmos directos reciben primero el patrón y luego el texto, y devuelven un contador de ocurrencias; Suffix Array se construye con el texto y se consulta con el patrón. Fuerza bruta se incorporó como línea base; KMP, Rabin–Karp y Suffix Array se obtuvieron del repositorio del curso. Las cuatro implementaciones cuentan coincidencias superpuestas. Para conservar los bytes originales del corpus se corrigió la interpretación de caracteres como `unsigned char` en los índices del Suffix Array y en el hash de Rabin–Karp, sin sustituir sus algoritmos.

Las implementaciones completas y los scripts experimentales se encuentran en la carpeta `Boletin_03/` del repositorio del estudio [1]; los algoritmos están en `include/string_matching/`.

3.1. Búsqueda por fuerza bruta

Esta implementación se agregó para el estudio. La función `brute_force` recibe vistas `string_view` del patrón y del texto. Un ciclo `for` prueba cada posición inicial válida, y un ciclo `while` compara los caracteres correspondientes. Si el patrón coincide completo, incrementa `matches`; al avanzar una posición en cada iteración, cuenta también coincidencias superpuestas.

3.2. Knuth–Morris–Pratt (KMP)

La función `kmp` construye el vector `p` de largo $m+1$, que guarda la información de prefijos y fallos del patrón. Un ciclo lo prepara; después, el ciclo de búsqueda usa `i` como índice del texto y `j` como índice del patrón. Ante una discrepancia, actualiza `j` con `p[j]` sin retroceder `i`; al completar una coincidencia, incrementa `matches` y reutiliza `p` para permitir solapamientos. La implementación proviene del repositorio del curso [3].

3.3. Rabin–Karp

La función `rk` usa `p` y `t` para los hashes del patrón y de la ventana actual, `q` como módulo y `d = 256` como base. Un ciclo inicial calcula ambos hashes; el ciclo principal recorre las ventanas. Cuando los hashes coinciden, un ciclo `while` verifica los caracteres y actualiza `count` si hay coincidencia. Después actualiza `t` en tiempo constante para la siguiente ventana. La implementación proviene del repositorio del curso [4].

3.4. Suffix Array

El constructor añade el centinela ETX al texto y crea el vector `SA` con las posiciones iniciales de los sufijos, además de los vectores `p`, `q` y `r` para posiciones temporales y clases. En cada ronda, un ciclo duplica el prefijo considerado; `count` acumula frecuencias para ordenar las clases. El método `count` busca con dos ciclos `while` los límites del intervalo de sufijos que comienza con el patrón, comparando con `substr` y `starts_with`; resta los límites para obtener el número de ocurrencias. La implementación proviene del repositorio del curso [2].

4. Metodología experimental

4.1. Características de la máquina

Los experimentos se realizaron en una estación de trabajo con las siguientes características:

- **Procesador:** Intel Core i5-13420H (Raptor Lake), 8 núcleos y 12 hilos (4 P-Cores y 4 E-Cores), frecuencia base de 2.10 GHz y turbo de hasta 4.60 GHz. Cachés: L1d de 320 KiB, L1i de 384 KiB, L2 de 7 MiB y L3 compartida de 12 MiB.
- **Memoria principal:** 16 GiB DDR (15 GiB utilizables por el sistema operativo).
- **Almacenamiento:** SSD NVMe WD PC SN740 de 512 GB, interfaz PCIe Gen4 $\times 4$.
- **Entorno de compilación:** Arch Linux x86_64 (kernel 6.x), GCC y las banderas `-std=c++20 -O3 -march=native`.
- **Afinidad de CPU:** se fijó el proceso de medición al núcleo lógico 2 mediante `taskset -c 2`. Esto reduce las migraciones entre núcleos, pero no elimina variaciones de frecuencia, temperatura ni la actividad concurrente del sistema.

4.2. Datos de entrada

Los textos procedieron del archivo `english.50MB` de Pizza & Chili, formado por textos en inglés de Project Gutenberg [5]. Se usaron sus primeros 10 millones de bytes y prefijos de 1, 2, 4 y 8 millones de bytes; en este estudio, $1\,\text{MB} = 10^6$ bytes. Las longitudes n y m se midieron en bytes, conservando mayúsculas, espacios y puntuación.

Los patrones fueron subcadenas del corpus, no necesariamente palabras completas. Antes de medir se verificaron sus frecuencias, incluidas las coincidencias superpuestas, y se conservaron sus bytes, longitudes y posiciones de extracción. El archivo `texts/ejemplos_experimentos.txt` incluye fragmentos reales y patrones usados, identificados según los manifiestos, con posiciones y conteos verificados en los 10 MB. Las posiciones parten de cero. Se conserva el archivo comprimido original y un manifiesto SHA-256; la preparación rechaza cambios de contenido. El prefijo verificado no contiene bytes de valor 0 a 3, por lo que se mantiene el centinela ETX del Suffix Array. Los bytes altos se interpretan sin signo, sin modificar el texto.

Como excepción obligatoria se generó $T = A^n$. Por ejemplo, `AAAAAAAAAA` contiene diez ocurrencias de $P =$ `A` y ninguna de $P =$ `B`.

4.3. Experimentos e hipótesis

Se compararon los cuatro métodos sobre las mismas entradas. La búsqueda de KMP y Rabin–Karp incluyó la preparación que sus funciones realizan en cada llamada; la consulta del Suffix Array usó el índice ya construido. Su construcción se midió por separado solo en el experimento del largo del texto, con 32 repeticiones por tamaño. En los otros experimentos y los casos unarios se construyó una sola vez para preparar las consultas, sin repetir esa evaluación. Así se informa el costo inicial

sin multiplicar mediciones del mismo trabajo. La lectura del archivo, la selección de patrones y la validación de conteos quedaron fuera del tiempo medido.

El esquema de medición se basó en la plantilla `uhr`, de Leonardo Lovera (LELE), usando `std::chrono::steady_clock` y nanosegundos [6]. Cada patrón se midió en un lote calibrado fuera de muestra para alcanzar al menos 5 ms o un máximo de un millón de consultas. El tamaño del lote se conservó entre repeticiones y su tiempo se dividió por la cantidad de consultas. La construcción se expresó en tiempo por construcción, sin incluir la destrucción del índice.

Cada configuración tuvo **32 repeticiones**, después del calentamiento. Se usaron los mismos lotes de patrones para todos los métodos; los conteos se consumieron para evitar su eliminación por el compilador y se validaron fuera del intervalo medido. Se aplicó afinidad al núcleo lógico 2 y una pausa de dos segundos entre series. Se conservaron las muestras crudas y se generaron CSV con **media, desviación estándar muestral, mínimo, Q1, mediana, Q3 y máximo**, calculados sobre los tiempos normalizados de las repeticiones. Las figuras muestran media y barras de una desviación estándar, no intervalos de confianza; el extremo inferior se limitó a cero. En el experimento de ocurrencias se conservó el resumen por patrón para no mezclar conteos distintos. Las repeticiones temporales no se trataron como patrones independientes. En Rabin–Karp se mantuvo $q = 101$ en todos los experimentos.

La memoria se midió en una pasada independiente mediante seguimiento de los bytes solicitados al asignador por cadenas y vectores. Se distinguieron el pico auxiliar por consulta, el índice persistente, el pico total de construcción y el pico temporal de las reservas liberadas durante la construcción. Se excluyen la entrada compartida, los metadatos del asignador y los datos del medidor; el seguimiento permanece desactivado al cronometrar. Fuerza bruta y Rabin–Karp no reservan memoria dinámica auxiliar, aunque usan variables escalares constantes. Estas medidas no representan la memoria residente total del proceso.

Para graficar frente a n y m, se promediaron con igual peso los tiempos de los patrones en cada repetición y se resumieron esos 32 promedios. Frente a z, cada patrón conservó su propio punto y dispersión. Se preservaron CSV crudos y resúmenes con los parámetros y hashes de cada sesión.

Los tamaños y cantidades de patrones se acotaron para procurar una duración computacional razonable, conservando 32 repeticiones y los tres efectos requeridos. No se presupone significancia estadística: se interpretaron las diferencias observadas junto con su dispersión. Las barras de desviación estándar no constituyen una prueba de hipótesis ni garantizan por sí solas diferencias significativas.

4.3.1. Efecto del largo del texto

- **Variable:** n, con textos de 1, 2, 4, 8 y 10 MB.
- **Controles:** cinco patrones distintos de $m = 32$ bytes, extraídos del primer MB y verificados como únicos en los 10 MB. Se usó exactamente el mismo conjunto en todos los tamaños, manteniendo $z = 1$. La selección se realizó antes de medir, sin insertar coincidencias.
- **Resultados:** tiempo por consulta frente a n, construcción del Suffix Array frente a n y consumo de memoria. Se generaron gráficos y tablas; los conteos coincidieron entre métodos.

Hipótesis. Con $|P|$ y el número de ocurrencias fijos, se esperaba que fuerza bruta y KMP aumentaran aproximadamente de forma lineal con el largo del texto. Se esperaba también un crecimiento cercano al lineal en Rabin–Karp si las colisiones eran escasas. La consulta del Suffix Array debería crecer más lentamente, de forma aproximadamente logarítmica, y la construcción del índice, como $n \log n$. Se esperaba un crecimiento lineal del índice y su memoria temporal con n, y memoria auxiliar aproximadamente constante en los otros métodos porque m no cambia.

4.3.2. Efecto del largo del patrón

- **Variable:** m, con longitudes de 8, 16, 32, 64, 128 y 256 bytes.
- **Controles:** un texto fijo de 10 MB y cinco patrones distintos por longitud, cada uno con exactamente una ocurrencia verificada. Así se evita confundir patrones más largos con una disminución del conteo. Se reutilizó el mismo Suffix Array para todas las longitudes.
- **Resultados:** tiempo por consulta y memoria frente a m. El costo fijo de construcción del índice se tomó del experimento del largo del texto, sin volver a cronometrarlo ni incluirlo en cada consulta.

Hipótesis. Se esperaba que KMP conservara un costo dominado por recorrer el texto, con preparación y memoria lineales en m, y que Rabin–Karp sumara una preparación lineal en m, pero no necesariamente una búsqueda más lenta si había pocas verificaciones. En fuerza bruta, el crecimiento de la cota de peor caso no implica un crecimiento lineal observado: muchas comparaciones pueden fallar pronto. El Suffix Array puede examinar prefijos más largos al aumentar m, con cota $O(m \log n)$, sin que aumente la memoria del índice.

4.3.3. Efecto del número de ocurrencias

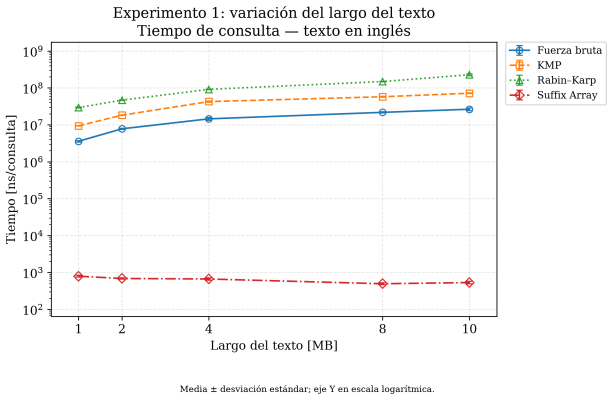
- **Variable:** z, el número de ocurrencias, incluidas las superpuestas. Se fijaron $n = 10$ MB y $m = 4$ bytes en el corpus inglés.
- **Selección:** se contaron las subcadenas distintas de cuatro bytes y se eligieron quince patrones presentes distribuidos entre frecuencias bajas, intermedias y altas. Se dividió el rango de frecuencias observadas en tres grupos sobre escala logarítmica y se tomaron cinco patrones por grupo. Se añadieron cinco patrones ausentes de igual longitud, cuya ausencia se comprobó. Las frecuencias reales, no valores impuestos, constituyeron el eje horizontal. El protocolo contemplaba reportar todos los disponibles y su cantidad si algún grupo no aportaba cinco patrones.
- **Caso extremo:** se usó además $T = A^n$, con los cinco tamaños del primer experimento, buscando $P = A$ y $P = B$. Ambos patrones tienen $m = 1$; se compararon $z = n$ y $z = 0$ por separado del corpus inglés.
- **Resultados:** tiempo por consulta frente a z, con n y m fijos en cada comparación. Los casos unarios se mostraron en otra figura, contrastando ambos patrones para cada tamaño.

Hipótesis. Se esperaba que Rabin–Karp tendiera a aumentar su tiempo con z, pues cada coincidencia real exige verificar los m caracteres, y que KMP siguiera siendo lineal en n, aunque podían variar sus constantes. La consulta del Suffix Array no enumera coincidencias, por lo que no se esperaba un costo proporcional a z. En fuerza bruta, más coincidencias implican más comparaciones completas, pero los intentos fallidos también influyen. En el caso unario con $m = 1$, se esperaba que fuerza bruta, KMP y Rabin–Karp siguieran siendo lineales en n: muchas coincidencias no bastan por sí solas para producir un costo cuadrático.

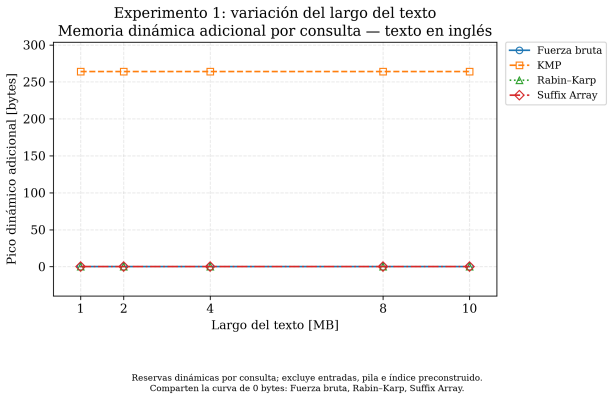
Consideración: en texto natural, patrones con frecuencias distintas también difieren en sus caracteres y prefijos. Este experimento estudia la relación observada entre frecuencia y tiempo, no un efecto causal aislado de z. Se usaron varios patrones por grupo para no depender de una única subcadena. La memoria auxiliar no debería variar con z, pues solo se devuelve un contador.

5. Resultados experimentales

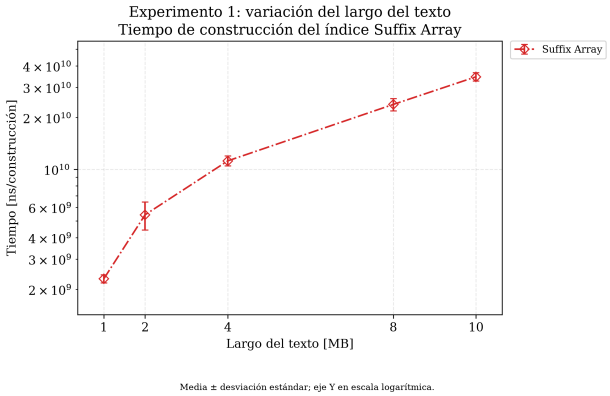
5.1. Experimento 1: largo del texto



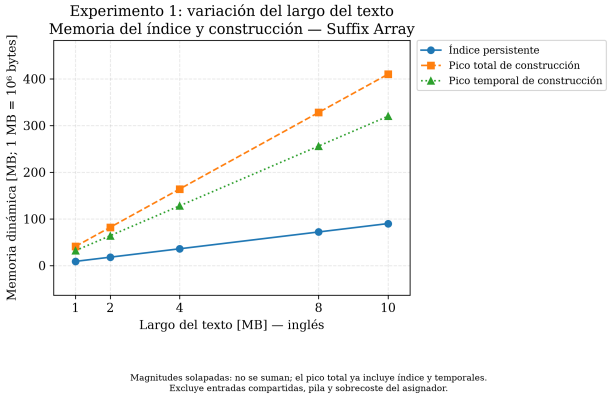
(a) Tiempo de consulta.



(b) Memoria auxiliar por consulta.



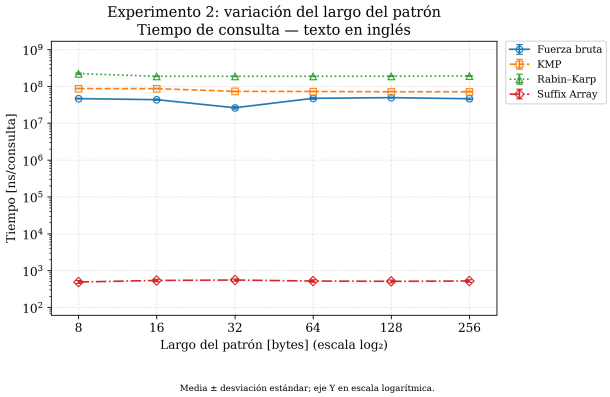
(c) Tiempo de construcción.



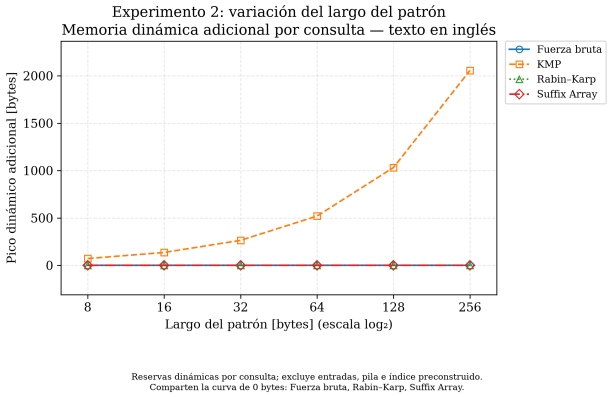
(d) Memoria del índice y construcción.

Figura 1: Experimento 1: largo del texto.

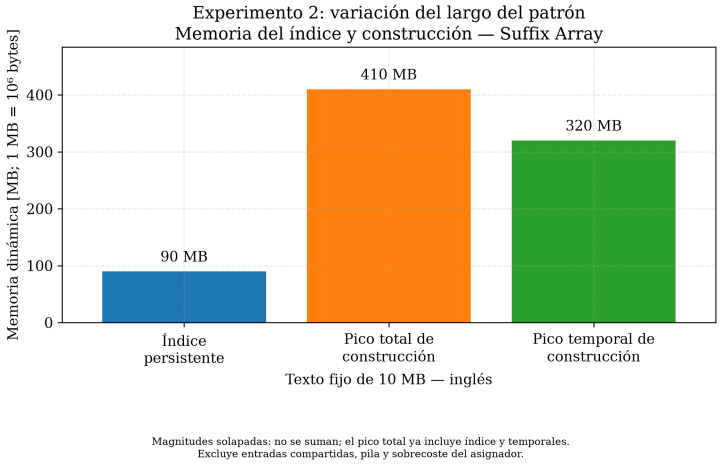
5.2. Experimento 2: largo del patrón



(a) Tiempo de consulta.



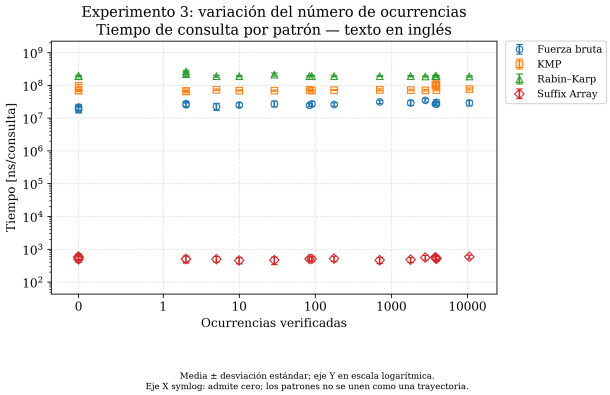
(b) Memoria auxiliar por consulta.



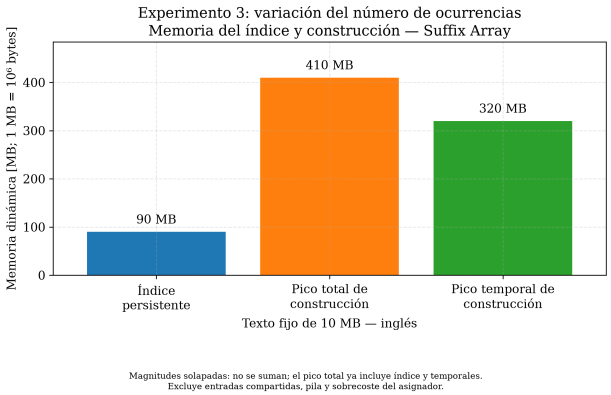
(c) Memoria del índice y construcción.

Figura 2: Experimento 2: largo del patrón.

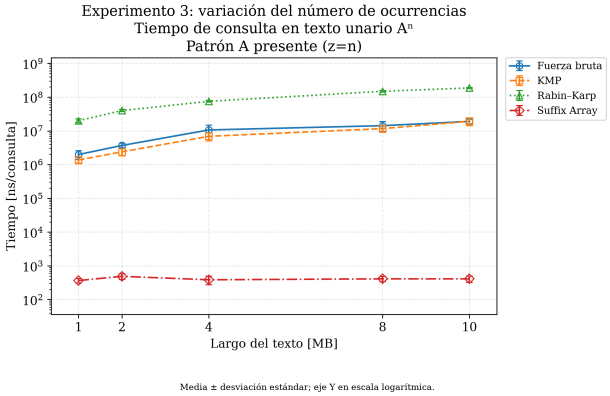
5.3. Experimento 3: número de ocurrencias



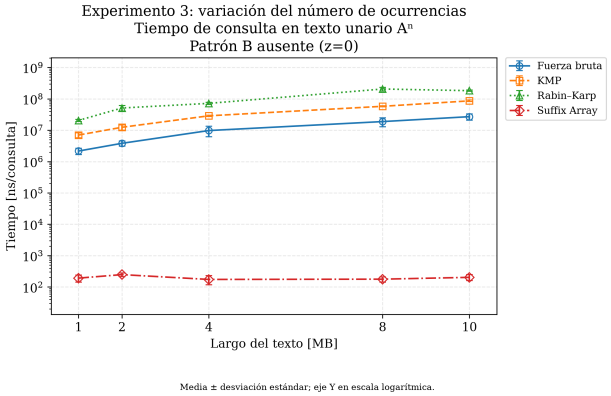
(a) Tiempo de consulta.



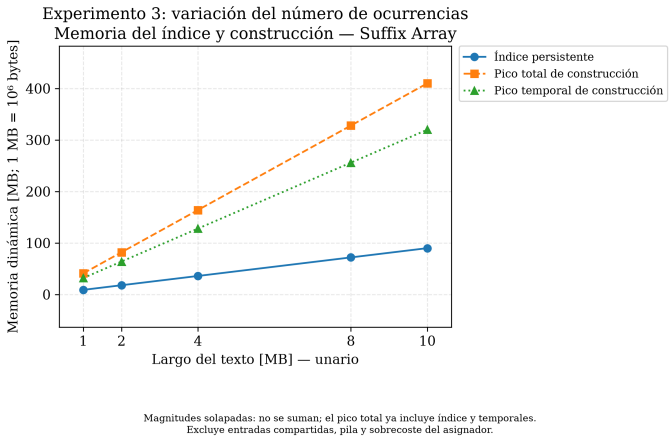
(b) Memoria del índice y construcción.



(c) Consulta de $P = A$ en texto unario.



(d) Consulta de $P = B$ en texto unario.



(e) Memoria del índice para texto unario.

Figura 3: Experimento 3: número de ocurrencias.

5.4. Resumen numérico de tiempos y memoria

Las tablas complementan los gráficos logarítmicos con los valores medidos. Los tiempos se expresan como media ± desviación estándar muestral de 32 repeticiones; no son intervalos de confianza.

Cuadro 1: Consulta según el largo del texto: patrones de 32 bytes y una ocurrencia.

Texto [MB]	Fuerza bruta [ms]	KMP [ms]	Rabin–Karp [ms]	Suffix Array [μs]
1	$3{,}59 \pm 0{,}17$	$9{,}38 \pm 0{,}17$	$29{,}30 \pm 0{,}99$	$0{,}790 \pm 0{,}047$
2	$7{,}80 \pm 0{,}34$	$18{,}25 \pm 0{,}25$	$46{,}68 \pm 0{,}77$	$0{,}685 \pm 0{,}029$
4	$14{,}54 \pm 1{,}23$	$42{,}63 \pm 1{,}88$	$91{,}52 \pm 1{,}44$	$0{,}663 \pm 0{,}044$
8	$21{,}78 \pm 0{,}95$	$57{,}60 \pm 1{,}99$	$148{,}96 \pm 2{,}63$	$0{,}493 \pm 0{,}030$
10	$26{,}47 \pm 2{,}15$	$71{,}86 \pm 1{,}95$	$227{,}43 \pm 12{,}68$	$0{,}532 \pm 0{,}046$

Cuadro 2: Consulta según el largo del patrón: texto de 10 MB y una ocurrencia.

Patrón [bytes]	Fuerza bruta [ms]	KMP [ms]	Rabin–Karp [ms]	Suffix Array [μs]
8	$46{,}32 \pm 1{,}61$	$86{,}75 \pm 2{,}04$	$221{,}30 \pm 10{,}67$	$0{,}488 \pm 0{,}044$
16	$43{,}37 \pm 1{,}83$	$86{,}42 \pm 2{,}37$	$187{,}39 \pm 3{,}32$	$0{,}537 \pm 0{,}044$
32	$26{,}18 \pm 2{,}20$	$72{,}94 \pm 1{,}93$	$186{,}65 \pm 4{,}20$	$0{,}552 \pm 0{,}043$
64	$47{,}23 \pm 2{,}89$	$72{,}34 \pm 1{,}63$	$187{,}00 \pm 3{,}23$	$0{,}517 \pm 0{,}042$
128	$49{,}24 \pm 2{,}08$	$71{,}13 \pm 1{,}79$	$188{,}07 \pm 4{,}34$	$0{,}509 \pm 0{,}045$
256	$45{,}97 \pm 2{,}26$	$71{,}23 \pm 2{,}74$	$190{,}37 \pm 4{,}78$	$0{,}519 \pm 0{,}043$

Cuadro 3: Construcción e índice persistente de Suffix Array sobre texto en inglés.

Texto [MB]	Construcción [s]	Índice persistente [MB]	Pico total de construcción [MB]
1	$2{,}30 \pm 0{,}12$	9,00	41,00
2	$5{,}43 \pm 1{,}02$	18,00	82,00
4	$11{,}20 \pm 0{,}74$	36,00	164,00
8	$23{,}86 \pm 1{,}97$	72,00	328,00
10	$34{,}57 \pm 1{,}94$	90,00	410,00

Notas. En las dos tablas de consulta se promedian primero, con igual peso, los cinco patrones de cada configuración dentro de cada repetición. La desviación estándar describe la variación de esos promedios, no la dispersión entre patrones. Las consultas de Suffix Array excluyen la construcción del índice. La memoria se observa en una pasada separada, sin desviación estándar; cuenta reservas dinámicas, no RSS ni pila. El pico total ya incluye la memoria persistente presente en ese instante: no se suman ambas columnas. Se usa $1\,\mathrm{MB} = 10^6$ bytes.

6. Conclusiones

6.1. Experimento 1: efecto del largo del texto

Al aumentar el texto de 1 a 10 MB, los tiempos de fuerza bruta, KMP y Rabin–Karp crecieron entre 7 y 8 veces, comportamiento compatible con un crecimiento aproximadamente lineal en el rango estudiado. Fuerza bruta presentó los menores tiempos entre los métodos sin índice sobre estas entradas, lo que muestra que una mejor cota de peor caso no garantiza un menor tiempo en cada conjunto de datos. La proporcionalidad no fue exacta; los resultados no permiten atribuir esas desviaciones a una causa concreta.

Las consultas de Suffix Array se mantuvieron por debajo de un microsegundo, sin crecimiento sostenido. Esto no demuestra tiempo constante ni que un texto mayor favorezca la búsqueda: su cota es $O(m \log n)$, y multiplicar el texto por diez cambia poco su logaritmo. Para 10 MB, la consulta promedió 0,532 μs, pero construir el índice requirió 34,57 s. El índice persistente ocupó 90 MB y el pico total de construcción alcanzó 410 MB; ambos crecieron linealmente con el texto. El pico ya incluye las reservas persistentes presentes en ese instante, por lo que no se suman ambas magnitudes. La ventaja de consulta tiene, por tanto, un costo inicial y espacial que debe evaluarse por separado.

6.2. Experimento 2: efecto del largo del patrón

Al extender el patrón de 8 a 256 bytes, no se observó un aumento sostenido del tiempo. Para KMP, esto es coherente con $O(n+m)$: frente a un texto fijo de diez millones de bytes, la preparación de patrones de hasta 256 bytes tiene un peso reducido. Rabin–Karp también recorre prácticamente la misma cantidad de ventanas. En fuerza bruta, muchas comparaciones pueden terminar antes de examinar el patrón completo, por lo que su cota $O(nm)$ no exige un crecimiento proporcional a m en estas entradas.

La caída observada para fuerza bruta con patrones de 32 bytes no permite concluir que esa longitud sea óptima: entre longitudes cambian también los patrones y sus prefijos. Suffix Array tampoco mostró una penalización sostenida al aumentar m, sin que ello invalide su cota teórica. En memoria, KMP sí incrementa la tabla auxiliar con el patrón, mientras el índice persistente permanece fijo porque el texto no cambia. La conclusión se limita, así, a una baja sensibilidad temporal en el rango evaluado.

6.3. Experimento 3: efecto del número de ocurrencias

En el corpus natural no apareció una relación proporcional clara entre frecuencia y tiempo. Los métodos sin índice recorren el texto incluso si el patrón está ausente; además, patrones con distintas frecuencias también difieren en contenido. Por ello, no puede atribuirse toda variación a z. Suffix Array localiza los límites del intervalo de coincidencias y los resta, sin visitar cada resultado: contar no exige el mismo trabajo que devolver todas sus posiciones.

Los casos $T = A^n$, con $P = A$ y $P = B$, contrastan presencia máxima y ausencia, pero no establecen que más coincidencias siempre implique mayor tiempo. Por ejemplo, para 10 MB KMP promedió aproximadamente 19 ms con $P = A$ y 86 ms con $P = B$. Al ser $m = 1$, los tres métodos sin índice mantienen costo lineal en n en ambos casos; estas pruebas no reproducen el peor caso cuadrático de fuerza bruta con patrones de longitud creciente.

6.4. Conclusión general

Los experimentos muestran que la selección de un método no debe basarse en una clasificación única de rapidez. Es necesario distinguir el costo de una consulta aislada del costo total de preparar y reutilizar una estructura, así como separar las garantías asintóticas del desempeño observado en un corpus concreto. Una curva casi horizontal en un gráfico logarítmico no demuestra complejidad constante, y una implementación con peor cota teórica puede resultar competitiva cuando las entradas no presentan su caso desfavorable.

Para pocas consultas o textos que cambian frecuentemente, los métodos sin índice evitan una construcción costosa y la reserva persistente asociada. Fuerza bruta constituye una alternativa sencilla y fue competitiva sobre el texto natural evaluado. Sin embargo, no ofrece la garantía $O(n+m)$ de KMP, que resulta preferible cuando interesa limitar el costo ante entradas desfavorables o repetitivas. Su mayor tiempo en parte de estos experimentos no elimina esa ventaja: el caso promedio observado y la protección frente al peor caso responden a necesidades distintas.

Para muchas consultas sobre un texto estático, Suffix Array puede compensar su construcción mediante búsquedas mucho más rápidas, siempre que haya memoria suficiente. Si Q es la cantidad de consultas, una comparación simplificada considera

$$T_{\mathrm{SA}}(Q) = T_{\text{construcción}} + Q\,t_{\mathrm{SA}}, \qquad T_{\text{sin índice}}(Q) = Q\,t_{\text{sin índice}}.$$

Este modelo ilustra la amortización, pero no determina un umbral universal: los tiempos dependen de los patrones, del equipo y de la posibilidad de reutilizar el índice. Tampoco representa una medición de una carga mixta de consultas o de actualizaciones del texto.

Rabin–Karp, con esta implementación y $q = 101$, no mostró una ventaja temporal frente a las otras alternativas evaluadas. No corresponde generalizar ese resultado a todas sus variantes ni atribuirlo únicamente a colisiones, cuyo número no se midió. Cambiar la configuración del hash y registrar las verificaciones permitiría estudiar mejor ese comportamiento.

El alcance de estas conclusiones está limitado por un solo corpus natural, los tamaños seleccionados y los casos sintéticos específicos. Las 32 repeticiones describen variabilidad temporal, pero no reemplazan la diversidad de entradas; sus desviaciones estándar tampoco prueban significancia estadística. Además, se midió conteo, no recuperación de posiciones, y memoria dinámica, no memoria total del proceso. Como extensión, sería útil evaluar otros corpus, patrones repetitivos de longitud creciente y cargas de muchas consultas que incluyan la construcción del índice.

En síntesis, no existe una alternativa universalmente superior. La decisión debe equilibrar cantidad de consultas, estabilidad del texto, memoria disponible y garantías requeridas, utilizando la teoría para anticipar riesgos y la experimentación para estimar los costos efectivos del escenario de uso.

Referencias

[1] Aguirre Fuentes, B. E. (2026). *EDAA-Experiments: Boletín 03, String Matching* [Código fuente]. https://github.com/StackDs/EDAA-Experiments/tree/main/Boletin_03.

[2] Implementación de Suffix Array, repositorio del curso EDAA. https://github.com/jfuentess/edaa/blob/main/strings/suffix_array.cpp.

[3] Implementación de KMP, repositorio del curso EDAA. https://github.com/jfuentess/edaa/blob/main/strings/kmp.cpp.

[4] Implementación de Rabin–Karp, repositorio del curso EDAA. https://github.com/jfuentess/edaa/blob/main/strings/rk.cpp.

[5] Repositorio de textos Pizza & Chili. http://pizzachili.dcc.uchile.cl/texts.html.

[6] Lovera, L. (LELE). (s. f.). *uhr: generic time performance tester* [Código fuente]. Plantilla de medición de la ayudantía, disponible en `utils/uhr.cpp` del repositorio. https://github.com/leonardlover.